CodeQuiz

Assignment 5: Advanced JavaScript and DOM Manipulation

Team	Members	Group
Team Spirit	Darkhan Saduakasov; Kassymkhan	SE 2508

Objective

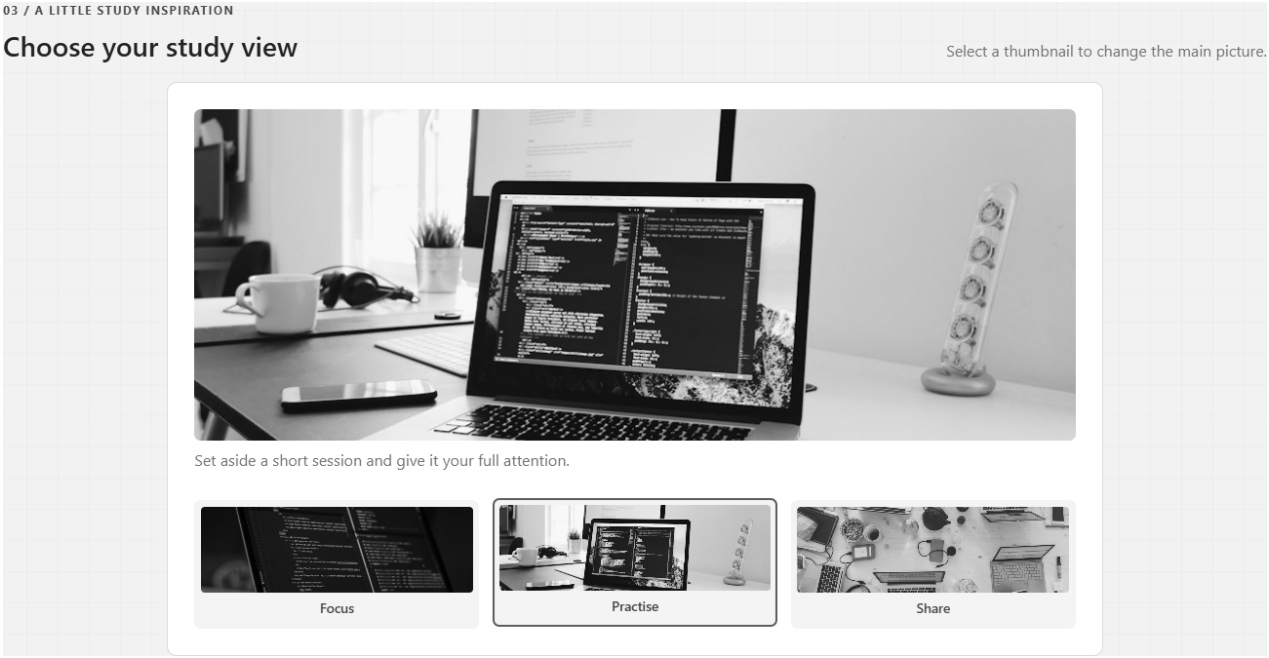
Extend the previous CodeQuiz website with DOM manipulation, event handling, objects, arrays, higher-order functions, sound and animations. Add a study planner, topic filters and a thumbnail gallery that support the programming quiz theme.

Website URL

1. DOM manipulation and styling

Steps

1. Select individual elements by ID and groups with querySelectorAll and jQuery selectors.
2. Update the plan summary, result text and available-topic count when the user interacts.
3. Change the background with the theme button and update visibility and selection styles.
4. For the thumbnail-gallery option, read the selected thumbnail attributes and change the main image src, alt and caption.



Gallery after selecting a thumbnail.

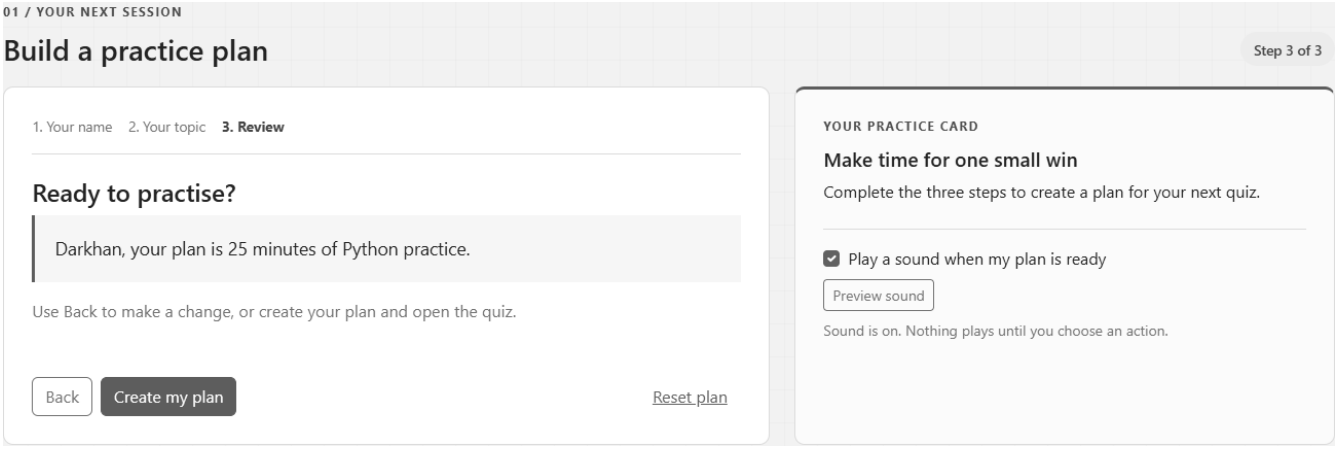
```
$(".study-thumb").click(function () {
    var source = $(this).attr("data-src");
    var description = $(this).attr("data-alt");
    var caption = $(this).attr("data-caption");
    $(".study-thumb").removeClass("selected").attr("aria-pressed", "false");
    $(this).addClass("selected").attr("aria-pressed", "true");
    $("#study-image").stop(true, true).fadeOut(120, function () {
        $(this).attr("src", source).attr("alt", description).fadeIn(160);
    });
    $("#study-caption").text(caption);
});
```

Code: changing image attributes, text and visibility.

2. Button events and callbacks

Steps

1. Handle the Next and Back buttons to move through the three form steps.
2. Pass updateStep to showStep as a callback to update progress and the visible controls.
3. Keep entered values in their fields when moving back and validate the current step before continuing.
4. Attach a native addEventListener click handler to Reset plan to clear fields, object properties and the result.



Review step with the selected name, topic and session length.

```
117     function showStep(number, callback) {
118         step = number;
119         $(".plan-step").stop(true, true).hide();
120         $("#plan-step-" + number).fadeIn(180);
121         callback(number);
122     }
123 

214     document.getElementById("plan-reset").addEventListener("click", function () {
215         document.getElementById("practice-plan").reset();
216         plan.name = "";
217         plan.topic = "";
218         plan.minutes = "15";
219         $("#practice-plan .is-invalid, #practice-plan .is-valid").removeClass("is-invalid is-
    valid").removeAttr("aria-invalid");
220         $("#practice-plan .invalid-feedback").text("").hide();
221         $("#plan-result-heading").text("Make time for one small win");
222         $("#plan-result").text("Complete the three steps to create a plan for your next quiz.");
223         $("#plan-quiz-link").hide().attr("href", "quiz.html");
224         $(".plan-result-panel").removeClass("completed");
225         audio.pause();
226         audio.currentTime = 0;
227         $("#sound-status").text("Plan reset. No sound is playing.");
228         showStep(1, updateStep);
229     });
230 
```

Code: step callback and reset event listener.

3. Keyboard events and switch

Steps

1. Select the menu links and attach keydown event listeners.
2. Find the focused menu item and use switch(event.key) to handle the arrow keys.
3. Move focus to the next or previous item and wrap at either end of the menu.
4. Use Enter to open the focused link. Apply a second switch to display the selected topic label.



Menu focus after arrow-key navigation.

```
function initMenuKeys() {
    var items = document.querySelectorAll("#mainNavigation .nav-link, #mainNavigation .trigger-modal-btn");
    for (var i = 0; i < items.length; i++) {
        items[i].addEventListener("keydown", function (event) {
            var next = 0;
            for (var j = 0; j < items.length; j++) {
                if (items[j] === this) next = j;
            }
            switch (event.key) {
                case "ArrowRight":
                case "ArrowDown":
                    next++;
                    break;
                case "ArrowLeft":
                case "ArrowUp":
                    next--;
                    break;
                case "Home":
                    next = 0;
                    break;
                case "End":
                    next = items.length - 1;
                    break;
                default:
                    return;
            }
            event.preventDefault();
            if (next < 0) next = items.length - 1;
            if (next === items.length) next = 0;
            items[next].focus();
        });
    }
}

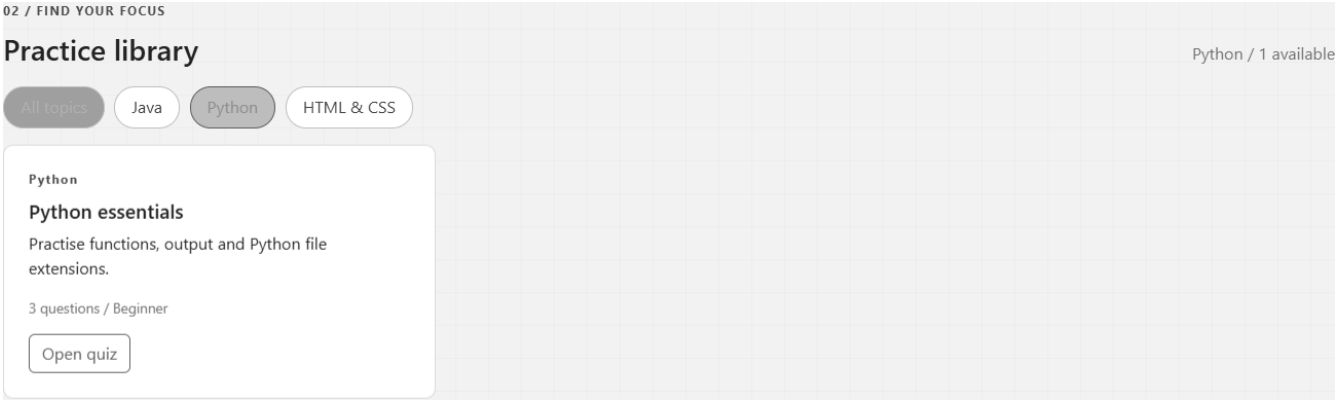
```

Code: keyboard listener, switch and focus movement.

4. Objects, arrays and higher-order functions

Steps

1. Store the learner name, topic and session length in the plan object.
2. Call the summary method to display those object properties in the review and result.
3. Store quiz topics in an array of objects and use a for loop to generate topic cards.
4. Pass matchesCategory to topics.filter() to display only the selected category. This is the higher-order function.



Filtered library showing the Python topic.

```
    var plan = {
        name: "",
        topic: "",
        minutes: "15",
        summary: function () {
            return this.name + ", your plan is " + this.minutes + " minutes of " + topicTitle(this.topic) + "
practice.";
        }
    };
```

```
    function matchesCategory(topic) {
        return currentCategory === "all" || topic.category === currentCategory;
    }

    function renderTopics(category) {
        currentCategory = category;
        var visibleTopics = topics.filter(matchesCategory);
        var container = $("#practice-topics");
        container.html("");
        for (var i = 0; i < visibleTopics.length; i++) {
            var topic = visibleTopics[i];
            container.append(
                '<div class="col-12 col-md-4"><article class="topic-card">' +
                '<p class="lab-eyebrow">' + topicTitle(topic.category) + '</p>' +
                '<h3>' + topic.title + '</h3>' +
                '<p>' + topic.description + '</p>' +
                '<p class="topic-meta">' + topic.questions + ' questions / Beginner</p>' +
                '<a class="btn btn-outline-primary" href="' + topic.link + '">Open quiz</a>' +
                '</article></div>'
            );
        }
        $("#topic-count").text(topicTitle(category) + " / " + visibleTopics.length + " available");
    }

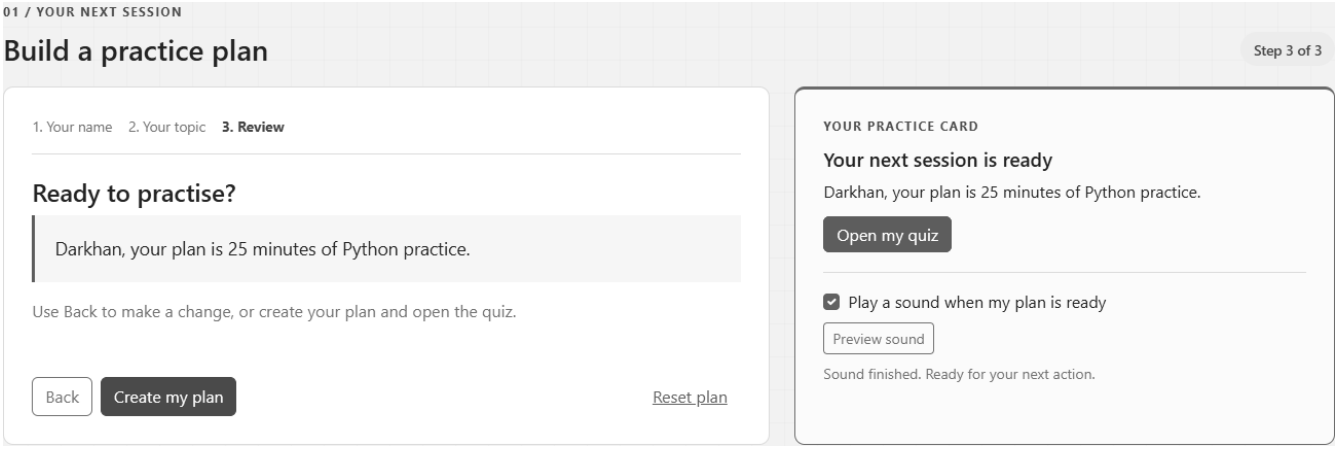
```

Code: object method, array, filter callback and loop.

5. Sound effects

Steps

1. Add one local notification sound to an audio element.
2. Play the sound when Preview sound is clicked or a practice plan is created.
3. Pause and rewind the same audio element before replaying it to avoid overlapping sounds.
4. Stop playback when sound is disabled or the plan is reset.



Completed plan with the sound controls.

```
167     function playSound() {
168         if (!document.getElementById("sound-enabled").checked) {
169             $("#sound-status").text("Sound is off.");
170             return;
171         }
172         audio.pause();
173         audio.currentTime = 0;
174         var playback = audio.play();
175         $("#sound-status").text("Playing the notification sound.");
176         if (playback) {
177             playback.catch(function (error) {
178                 if (error.name !== "AbortError") {
179                     $("#sound-status").text("The browser could not play audio. Check sound permissions and try
    again.");
180                 }
181             });
182         }
183     }
184
185     $("#test-sound").click(function () { playSound(); });
```

Code: sound option, pause, rewind and playback.

6. Event-triggered animations

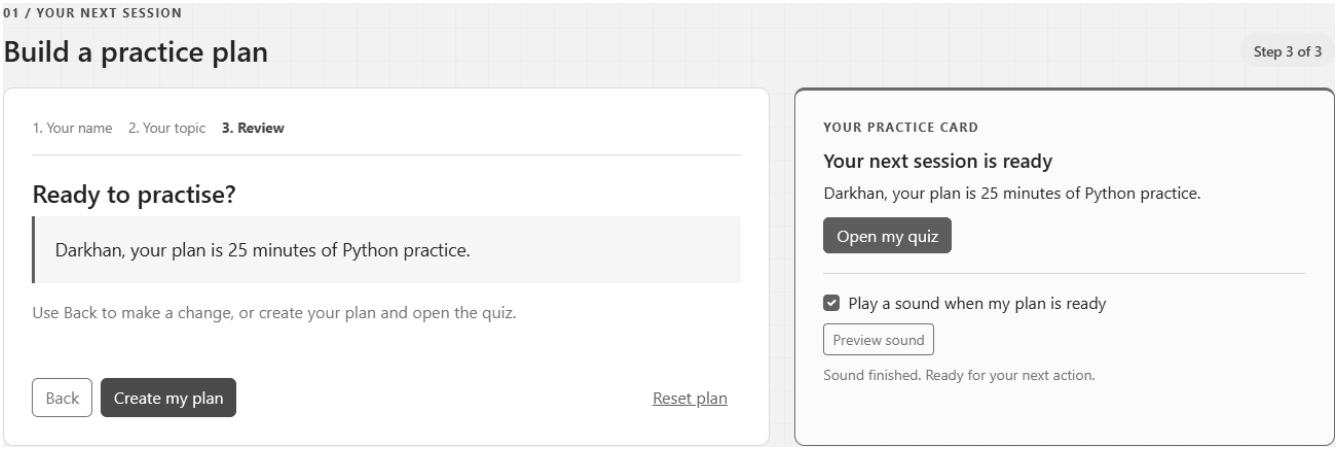
Steps

1. Use fadeIn when moving between planner steps.
2. Fade the gallery image out, change its attributes in the callback and fade it back in.
3. Use css and animate to fade the completed-plan result to full opacity.
4. Use stop(true, true) before starting another animation so rapid actions do not build up a queue.

```
    $("#practice-plan").submit(function (event) {
        event.preventDefault();
        if (step < 3) {
            if (checkStep()) showStep(step + 1, updateStep);
            return;
        }
        $("#plan-result-heading").text("Your next session is ready");
        $("#plan-result").text(plan.summary());
        for (var i = 0; i < topics.length; i++) {
            if (topics[i].category === plan.topic) $("#plan-quiz-link").attr("href", topics[i].link).show();
        }
        $(".plan-result-panel").addClass("completed").stop(true, true).css("opacity", 0.4).animate({ opacity: 1 },
250);
        playSound();
    });

    $(".study-thumb").click(function () {
        var source = $(this).attr("data-src");
        var description = $(this).attr("data-alt");
        var caption = $(this).attr("data-caption");
        $(".study-thumb").removeClass("selected").attr("aria-pressed", "false");
        $(this).addClass("selected").attr("aria-pressed", "true");
        $("#study-image").stop(true, true).fadeOut(120, function () {
            $(this).attr("src", source).attr("alt", description).fadeIn(160);
        });
        $("#study-caption").text(caption);
    });
```

Code: event handlers triggering fades and the result animation.



Result after the completion animation.

7. Structure and user experience

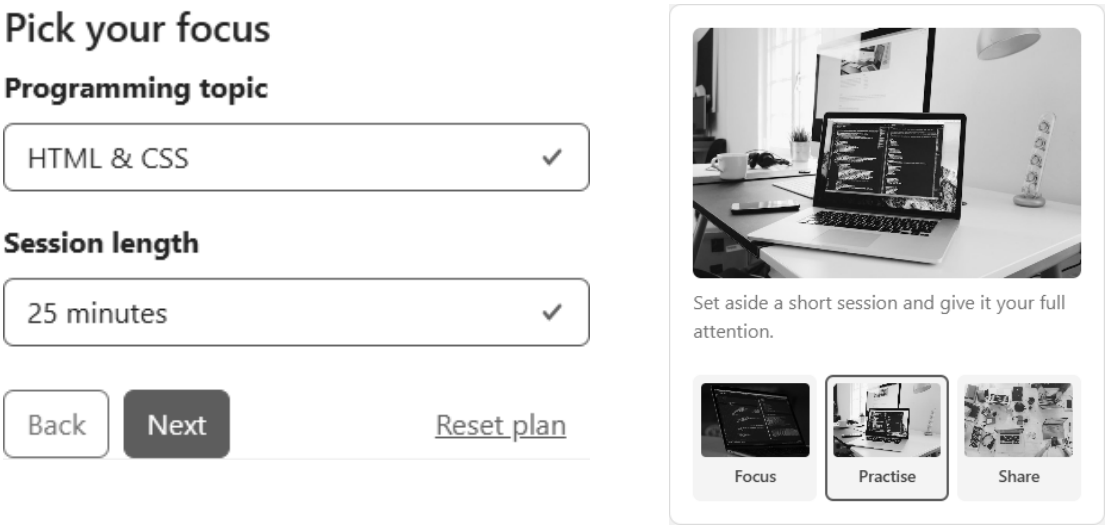
Steps

1. Keep the page structure in HTML, presentation in separate CSS files and behaviour in separate JavaScript files.
2. Load jQuery before the custom scripts and use named functions for repeated actions.
3. Use labelled form fields, visible feedback, menu keyboard navigation and responsive layouts.
4. Keep the new planner, filters, gallery and sound controls connected to the CodeQuiz learning theme.

```
10      <link href="vendor/bootstrap/bootstrap.min.css" rel="stylesheet">
11      <link rel="stylesheet" href="css/style.css">
12      <link rel="stylesheet" href="css/assignment5.css">

269     <script src="vendor/jquery/jquery-3.7.1.min.js"></script>
270     <script src="vendor/bootstrap/bootstrap.bundle.min.js"></script>
271     <script src="js/script.js"></script>
272     <script src="js/assignment5.js"></script>
```

Code: separate styles and scripts in the correct loading order.



Planner and gallery on a narrow screen.

Reflection

We connected the new study planner and topic filters to the existing quizzes. Keeping input values while moving between form steps was a challenge; hiding panels and using a shared plan object preserved the data. Reusing one audio element prevented sounds from overlapping. This work helped us practise callbacks, objects and DOM updates.